

概率条件并行 DAG 任务的响应时间分析与优化

Niklas Ueter
TU Dortmund University
Dortmund, Germany
niklas.ueter@tu-dortmund.de

Mario Günzel
TU Dortmund University
Dortmund, Germany
mario.guenzel@tu-dortmund.de

Jian-Jia Chen
TU Dortmund University
Dortmund, Germany
jian-jia.chen@tu-dortmund.de

摘要—实时系统日益使用多核处理器，以满足热、功耗和计算需求。为了利用多核处理器提供的架构并行性，必须提供满足实时约束的并行任务模型、调度算法和响应时间分析。本文针对具有概率执行行为的偶发、受限截止时间并行条件 DAG 任务，提出基于资源预留的调度算法，适用于能够容忍有限截止时间错过次数和有限延误的应用。我们制定设计规则与分析方法，在不强制立即中止超期作业的情况下，为连续 k 次错过截止时间的概率提供指定上界，并保证延误有界。

关键词—实时调度，分布式计算，并行任务模型

I. 引言

实时系统中的截止时间错过可能导致灾难，因此必须形式化验证系统的时间行为，以保证安全。过去十年，实时系统从单处理器转向多处理器，以应对现代复杂应用的计算、热和能耗约束。为此，大量研究探讨如何利用多处理器并行性，为具有任务间和任务内并行性的任务集满足截止时间约束。任务间并行性指不同串行任务可以并发执行，任务内并行性则指一个任务本身允许并行执行。相关研究提出了 fork/join 模型 [18]、同步并行任务模型及其实时调度算法和响应时间分析，例如 [29]，以及基于 DAG，即有向无环图的任务模型 [2], [6], [14], [15], [25]。这些模型使计算机视觉、雷达跟踪和视频等执行需求较高且具有内在并行性的任务，能够在更紧的截止时间下调度。

除上述模型表示任务内并行性的不同方式及理由外，正如 Melani 等人 [25] 所述，自动驾驶和图像处理领域的并行应用还存在多个条件分支和控制流指令。此外，这些领域并行算法的子作业执行时间会随传感器输入显著变化，例如自动驾驶车辆中用于目标检测的图像。不仅如此，多核架构已被证明会使最坏情况时间分析复杂化，因为缓存、内存等共享资源的竞争会产生干扰效应。文献 [13] 指出，资源共享导致的仲裁延迟和状态扰动必须纳入最坏情况界。这些不确定性最终使实时系统响应时间分析过于悲观，从而导致资源利用不足。研究已充分考察架构对最坏执行时间分析的影响，例如缓存分区 [1] 和内存访问的带宽共享机制 [34]。

另一种思路是接受并行任务不确定的执行行为，并关注概率响应时间特征。对于闭环反馈控制器等许多应用，由于控制器本身能鲁棒应对抖动、截止时间错过等时间非理想性，因此无需采用具有安全但极其悲观上界的硬实时系统工程。事实上，只要控制应用错过截止时间的次数有限，仍可以满足所要求的控制质量。

近期，许多研究着眼于形式化和分析放宽的截止时间约束 [28]，例如弱硬实时系统，要求 k 个任务实例中至少 m 个满足截止时间。此外，Maggio 等人 [22] 研究了连续截止时间错过约束下闭环控制系统的稳定性，进一步说明需要为应用提供能够保证连续截止时间错过概率界的调度算法。

为了形式化描述和验证截止时间错过的定量保证，软实时系统需要关注几个量：一次截止时间错过的概率、连续 k 次截止时间错过的概率，以及作业的最大延误。尽管保证是软性的，精确量化截止时间错过仍十分困难；即使对于单处理器上的普通串行实时任务模型，也充满挑战。第 II 节总结了这一方向的文献。现有分析只能在严格模型假设下推导上述量，例如一旦作业超过截止时间就将其中止。复杂性的部分原因在于任务间干扰，即高优先级作业造成的抢占与干扰模式，使响应时间分析必须考虑大量系统状态。

我们的目标是在同构多处理器上，分析、优化和验证概率条件并行 DAG 任务的可调度性，涉及截止时间错过概率、连续截止时间错过概率及延误约束等量。考虑概率并行 DAG 任务的调度与分析时，不仅要考虑任务间干扰，还必须考虑任务内干扰以及多处理器调度异常，即作业提前完成反而可能导致更长响应时间；这些因素使上述量的分析进一步复杂化。

贡献：我们提出基于资源预留，即服务供给的调度算法，对并行 DAG 任务进行概率分析，以避免任务间干扰引起的复杂性与调度异常，从而首次能够实现所述目标。具体贡献如下：

- 在第 III 节中，我们为广泛使用的条件并行 DAG 任务模型提出概率版本及形式化描述。
- 我们提出基于资源预留的概率条件并行 DAG 任务调度算法和响应时间分析。资源预留可以使用任意现有调度范式，与其他实时工作负载共同调度。此外，我们提供设计规则，构造能够保证有界延误、稳定性以及连续 k 次错过截止时间概率上界等概率特征的资源预留。由于资源预留和工作负载模型抽象能够处理调度或动态 DAG 结构引起的提前完成，我们的方法不会产生调度异常。

据我们所知，这是首篇针对概率条件并行 DAG 任务集，提供定量保证的分析与优化论文。

II. 相关工作

在多处理器上调度具有最坏情况参数，例如最坏执行时间的并行实时任务，已经针对不同并行任务模型得到广泛研究。Goossens 等人 [16] 较早将具有实时约束的并行任务分为刚性 (rigid)、可塑 (moldable) 或可变 (malleable) 任务。早期并行任务模型研究关注同步并行模型，例如 [11], [23], [29]。同步模型扩展了 fork-join 模型 [12]，允许每个同步片段具有不同数量的子任务，且子任务数量可以超过处理器数量。许多已提出的调度算法和分析基于分解，即将并行任务分解为一组串行任务并调度。

近期，有向无环图 (DAG) 任务模型被提出，并成为调度算法设计与分析的研究对象。DAG 任务具有更一般的

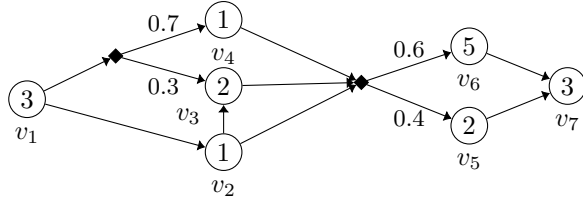


图 1. 一个概率条件 DAG 任务示例, 其中每个条件节点, 即菱形, 表示运行时只释放一个相邻子作业, 选择概率标注在边上。在此例中, 运行时可以实例化四种不同 DAG 结构。

并行结构: 每个任务由一组子任务及其先后约束描述, 并用有向无环图表示。Melani 等人 [31] 和 Sun 等人 [32] 表明, 这种并行模型对应于 OpenMP 等并行计算 API 的模型。已有研究考察了全局调度, 例如 [6], [26], 以及分区调度算法 [8], [15]。对于同步任务和一般 DAG 任务, 还存在不基于分解的方法, 例如 Li 等人 [21] 提出的联邦调度, 避免并行任务的作业间干扰。联邦调度将 DAG 任务划分为两类: 可以在单个处理器上串行执行并满足截止时间的任务, 以及必须并行执行才能满足截止时间的任务; 后一类任务独占分配给它们的一组处理器。

受自动驾驶、计算机视觉等现代并行应用条件执行行为的启发, 研究提出了条件 DAG 任务模型。大量工作研究该模型的实时可调度性, 例如 [3], [10], [25]。近期, Marchetti 等人 [24] 考察了具有实时约束的条件 DAG 调度的计算复杂度。然而, 在形式化验证调度算法时, 必须考虑最坏情况参数和最坏条件结构, 因而不可避免地过度供给资源。

对于能够容忍有限截止时间错过次数的软实时应用, 这类并行任务的概率模型和响应时间分析具有研究价值。此外, 在并行架构上推断最坏情况参数越来越复杂、悲观, 进一步凸显了概率模型和分析的重要性。串行随机任务已有大量概率分析工作, 例如 [17], [30]。近期工作关注提高基于卷积的截止时间错过概率分析效率。Brüggen 等人 [7] 利用多种状态空间缩减技术、基于 Hoeffding 和 Bernstein 不等式的近似, 以及等价类合并, 提出对多项分布的高效卷积。Chen 等人 [9] 使用 Chebyshev 不等式、矩生成函数及其优化, 高效计算连续截止时间错过。研究也尝试用预留服务器调度概率串行任务。例如, Palopoli 等人 [27] 展示如何计算使用恒定带宽服务器 (CBS) 调度的周期实时任务错过截止时间的概率; 他们将问题归约为具有周期结构的无限状态离散时间 Markov 链稳态概率计算。在并行 DAG 任务方面, Ueter 等人提出资源预留方案, 以调度具有实时约束的偶发任意截止时间 DAG 任务 [33]。另一种概率实时任务分析方法, 是 Lehoczký 等人 [19] 提出的实时排队理论, 将经典排队理论扩展到具有截止时间的系统。Li [20] 较早研究并行 DAG 任务的概率响应时间, 并扩展联邦调度工作 [21], 使用排队理论设计联邦调度参数, 使每个任务延误有界, 并满足软实时要求。更近的工作是 Ben-Amor 等人 [4], [5], 研究在多处处理器上, 以子任务为粒度采用分区固定优先级调度时, 并行 DAG 任务的概率响应时间。其模型为每个子任务给出概率最坏执行时间及静态先后约束, 再通过卷积方法推导子任务响应时间概率, 并组合为总体响应时间。

概率	长度	总工作量
0.42	12	13
0.18	13	14
0.28	9	10
0.12	11	11

表 I

图 1 中概率条件 DAG 任务的总工作量与长度参数概率的表格表示。

III. 任务与问题模型

考虑由 M 个相同, 即同构处理器组成的多处理器系统, 以及给定的概率偶发、受限截止时间条件并行有向无环图 (DAG) 任务集 $\mathbb{T}$ 。每个任务释放无限个任务实例, 称为作业。每个条件并行 DAG 任务 $\tau_i \in \mathbb{T}$, 由条件 DAG 结构 G_i 、相对截止时间 D_i 以及最小到达间隔 T_i 定义, 结构稍后说明。最小到达间隔表示两个作业释放之间的最短距离。本文只考虑受限截止时间任务, 即对每个任务 τ_i , 都有 $D_i \leq T_i$ 。图 1 展示一个概率条件 DAG 示例。该图由节点 V 和边 E 组成, 表示先后与控制流约束。每个节点要么是具有执行时间的子作业节点, 要么是表示向子作业进行概率条件分支的条件节点。示例中有两个决策节点, 各有两个分支选项, 因此产生四种可枚举 DAG 实现; 某个实现的概率由经过相应条件节点路径的概率给出。条件 DAG 由有限个 DAG 构成, 每个 DAG 是二元组 (V, E) , 其中 V 是有限子作业集合, 关系 $E \subseteq V \times V$ 表示子作业先后约束, 且底层图不存在有向环。对于每个 DAG, 其总工作量和长度参数计算如下。

记 $pre(v_i) := \{v_j \in V \mid (v_j, v_i) \in E\}$ and $v_j \prec v_i$ if $v_j \in pre(v_i)$ 相应地, 记 $succ(v_i) := \{v_j \in V \mid (v_i, v_j) \in E\}$ and $v_j \succ v_i$ if $v_j \in succ(v_i)$ 。

定义 1 (路径). 有向无环图 G 中的路径 π , 是满足 $v_{i_j} \in V$ 的任意子作业序列 $v_{i_1} \prec v_{i_2} \prec \dots \prec v_{i_k}$, 且 $pre(v_{i_1}) = \emptyset$, $succ(v_{i_k}) = \emptyset$ 。

定义 2 (长度). 设路径 π 为一个子作业序列, 按先后约束, 序列中的每个子作业都是前一个子作业的直接后继, 则路径长度为

$$\text{len}(\pi) := \sum_{v_i \in \pi} \text{len}(v_i)$$

其中, 子作业的长度表示其执行时间。由此, DAG G 的长度定义为

$$\text{len}(G) := \max\{\text{len}(\pi) \mid \pi \text{ is a path in } G\}.$$

定义 3 (总工作量). DAG G 的总工作量由图中全部执行时间的累加给出, 即

$$\text{vol}(G) := \sum_{v_i \in V} \text{len}(v_i).$$

A. 概率参数描述

每个概率条件 DAG 任务用三元组 $\tau_i = (G_i, D_i, T_i)$ 描述, 其中 G_i 为概率条件 DAG 结构, D_i 为相对截止时间, T_i 为两个作业释放之间的最小到达间隔。对于每个任务 $\tau_i \in \mathbb{T}$, 从条件 DAG 结构推导累积分布函数 (CDF), 其中 $F_i(u, v)$ 描述 DAG 实例的总工作量和长度的概率行为。也就是说, 每个任务 τ_i 释放无限个作业 $\tau_{i,\ell}$, $\ell = 0, 1, 2, \dots$; 每个作业对应 DAG 实例 $G_{i,\ell}$, 其总工作量与长度参数, 是按分布函数的概率描述获得的 $G_{i,\ell}$ 的一次实现。

例如, 对于图 1 中的条件 DAG, 计算每个 DAG 实现的概率和对应参数值, 即可得到分布函数。图 2 的实例选择了两条上方边, 出现概率为 $0.7 \cdot 0.6 = 0.42$, 对应长度为 12, 总工作量为 13。同理, 选择概率为 $0.7 \cdot 0.4 \cdot 0.3 \cdot 0.6$ 和 $0.3 \cdot 0.4$ 的边, 得到对应 DAG 结构的实现概率 0.28、0.18 和 0.12。计算每种实现的总工作量与长度, 即可得到表 I 中的数据。

由此, 推导 $F_i(u, v) = \mathbb{P}(\text{vol}(G_i) \leq u, \text{len}(G_i) \leq v)$ 如下:

$$\begin{aligned} & 1(u-13) \cdot 1(v-12) \cdot 0.42 + 1(u-14) \cdot 1(v-13) \cdot 0.18 \\ & + 1(u-10) \cdot 1(v-9) \cdot 0.28 + 1(u-11) \cdot 1(v-11) \cdot 0.12 \end{aligned}$$

其中, 1 为阶跃函数, 即当 $x \geq 0$ 时, $1(x)$ 为 1, 否则为 0。对于所介绍的概率条件 DAG 任务, CDF 是具有有限阶梯的阶跃函数。此外, 我们假设 DAG 实例相互独立。

B. 延误

系统必须处理每个错过截止时间的作业, 即制定机制决定此类事件发生后的动作。常见机制是立即中止超过截止时间的作业, 以避免干扰后续作业。这种做法效率较低, 因为所有计算结果与状态变化都会丢弃, 甚至为了一致性还需要回滚; 尤其当超过截止时间的幅度很小时, 更不合理。非形式化地说, 作业延误度量它相对于截止时间的延迟。

定义 4 (延误). 设 $\delta_i(\ell)$ 表示任务 τ_i 的第 ℓ 个作业的延误, 即考虑此前作业可能存在积压工作量时, 第 ℓ 个作业超过任务截止时间的时长。延误可以递归表示为 $\delta_i(\ell) = \max\{\delta_i(\ell-1) + (R_{i,\ell} - D_i), 0\}$, 其中 $R_{i,\ell}$ 是任务 τ_i 的第 ℓ -th 个作业的响应时间。此外, 按定义 $\delta_i(0) = 0$ 。

按该定义, 若 $\delta_i(\ell) = 0$, 则任务 τ_i 的第 ℓ 个作业满足截止时间; 若 $\delta_i(\ell) > 0$, 则错过截止时间。为改善这一问题, 我们拟通过延误上界限制任务每个作业的延误。

定义 5 (延误上界). 若任务 τ_i 的任意作业在延误超过 ρ_i 时被中止, 则称其具有延误上界 $\rho_i > 0$, 即对所有 $\ell \geq 0$, 均有 $0 \leq \delta_i(\ell) \leq \rho_i$ 。

延误上界由用户指定, 使概率偶发、受限截止时间并行 DAG 任务的形式化描述进一步成为四元组 (F_i, D_i, T_i, ρ_i) 。

C. 截止时间错过

我们的目标是设计资源预留系统, 为任务集 $\mathbb{T} = \{\tau_1, \tau_2, \dots, \tau_n\}$ 中的每个任务 τ_i 提供充分服务, 使连续 k 次错过截止时间的概率有界。

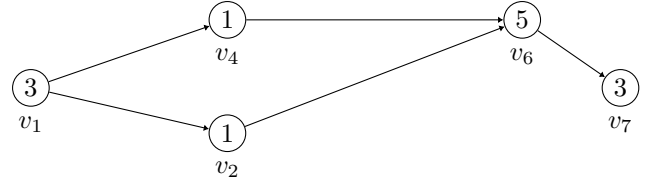


图 2. 图 1 中示例条件 DAG 任务的一次 DAG 实现, 选择概率为 0.7 和 0.6 的条件分支。

定义 6 (连续截止时间错过). 对于 $\ell \geq 0$, 任意连续释放的 k 个作业序列 $\tau_{i,\ell}, \tau_{i,\ell+1}, \dots, \tau_{i,\ell+k-1}$, 在满足以下条件时, 称为连续 k 次错过截止时间:

- 序列中的全部作业均错过截止时间。
- $\ell = 0$, 或者前一个作业 $\tau_{i,\ell-1}$ 未错过截止时间。 ■

对于每个任务, 定义函数 $\theta_i: \mathbb{N} \rightarrow [0, 1]$, 表示以不超过 $\theta_i(k)$ 的给定概率, 容忍连续 k 次错过截止时间。

定义 7 (连续 k 次截止时间错过约束). 令 $\phi_i(j, k) := \mathbb{P}(\delta_i(j) > 0, \dots, \delta_i(j+k-1) > 0 \mid j = 0 \text{ or } \delta_i(j-1) = 0)$ 表示作业序列 $\tau_{i,j}, \tau_{i,j+1}, \dots, \tau_{i,j+k-1}$ 连续 k 次错过截止时间的概率。若满足下式, 则称概率条件 DAG 任务 τ_i 满足截止时间约束 $\theta_i(k)$:

$$\sup_{j \geq 0} \{\phi_i(j, k)\} = \phi_i(0, k) \leq \theta_i(k), \quad (1)$$

即在每个位置 j , 概率 $\phi_i(j, k)$ 不超过阈值 $\theta_i(k)$ 。 ■

式 (1) 中的等式, 来自作业 $\tau_{i,j}$ 释放之前不存在积压工作量这一条件。

IV. 调度问题

我们用资源预留系统处理 DAG 任务调度, 并使用任意分区调度算法, 调度该预留系统与系统中的其他任务。

A. 资源预留

在资源预留系统中, 按某种规则为每个概率并行 DAG 任务 τ_i 预留服务; 任务 τ_i 的实例可以在这些预留服务上处理。若同一时刻最多存在 $m_i \in \mathbb{N}$ 个预留, 则称该预留系统具有 m_i 路并行性。本文考虑这种并行预留系统的简化版本:

定义 8 (本文的资源预留系统). 一个资源预留系统包含 m_i 个预留服务器, 每个提供服务量 E_i , 并每隔 $P_i > 0$ 个时间单位补充预算。具体而言, 为提供服务, 每隔 P_i 个时间单位, 激活一个由 $m_i \in \mathbb{N}$ 个不同预留组成的多重集合; 每个预留保证在长度为 P_i 的区间内, 提供 E_i 个时间单位的服务。

任务实例按先入先出 (FIFO) 方式分配给所提供的服务。此外, 我们假设每个时刻, 所有已分配预留按照 FIFO 策略, 只服务同一个 DAG 作业的子作业。

预留系统使用任意调度范式, 在 M 个同构处理器上调度; 每当获得执行机会, 就按以下方式为 DAG 作业提供服务。

定义 9 (列表调度). 在 m_i 路并行预留服务器上的列表调度中, 对于给定 DAG 作业 $G = (V, E)$, 一旦

某个子作业的全部前驱子作业执行完毕，就在任意处于空闲且已获调度执行的预留服务器上运行该子作业。更形式化地说，每个子作业 v_i 的开始时刻 s_i 为 $\min\{t \mid \text{some scheduled reservation server idles at } t, t \geq \max\{f_j \mid v_j \in \text{pre}(v_i)\}\}$ 。

本节后续假设，在 M 个同构处理器上存在一个可行调度 S ，意味着所有资源预留都提供承诺的服务。

定义 10 (完成工作量). 设给定调度 S , $work_i^S(t_1, t_2)$ 表示从任务 τ_i 产生的 DAG 作业在时间区间 t_1 到 t_2 内实际完成的工作量。

根据这一定义，DAG 任务 τ_i 在 $t_{i,\ell}$ 释放的作业 $\tau_{i,\ell}$ ，其最坏情况响应时间由满足 $work_i^S(t_{i,\ell}, t') \geq vol(G_i^\ell) + backlog_i^S(t_{i,\ell})$ 的最小 $t' \geq t_{i,\ell}$ 给出；其中 $backlog_i^S(t_{i,\ell})$ 是时刻 $t_{i,\ell}$ ，任务 τ_i 在 $t_{i,\ell}$ 之前释放的作业中尚未完成的工作量。若此前没有截止时间错过，则 $backlog_i^S(t_{i,\ell}) = 0$ ，因为系统模型假设 $D_i \leq P_i$ 。下面将处理的工作量用所供给服务表示，并推导定理 1 中的响应时间界。

为论证方便，设 S 是资源预留系统的一个可行调度，将 DAG 任务 τ_i 的一个作业执行至完成。再设 $serv_i^S(t_1, t_2)$ 表示调度 S 在时间区间 t_1 到 t_2 内，为该 DAG 作业提供的服务。

定义 11 (包络). 设 S 是任务集 $\mathbb{T}$ 的一个具体调度。考虑 $\mathbb{T}$ 中某个任务的 DAG 作业实例 G ，其子作业集合为 $V = \{v_1, \dots, v_\ell\}$ 。每个子作业 v_k 在 S 中的开始时刻为 s_k ，完成时刻为 f_k 。我们通过以下性质递归定义 G 的包络 $s_{k_1}, f_{k_1}, s_{k_2}, f_{k_2}, \dots, s_{k_p}, f_{k_p}$ ，其中 $p \in \{1, \dots, \ell\}$ ：

- 1) 对所有 $i \neq j$ ，有 $k_i \neq k_j \in \{1, \dots, \ell\}$ 。
- 2) v_{k_p} 是 V 中完成时刻最大的子作业。
- 3) 对所有 $i \in \{p, p-1, \dots, 2\}$ ， $v_{k_{i-1}}$ 是 $\text{pre}(v_{k_i})$ 中完成时刻最大的子作业。
- 4) $\text{pre}(v_{k_1}) = \emptyset$

若存在完成时刻相同的子作业，DAG 作业实例的包络定义可能不唯一；此时任意选择一个即可。

根据包络定义，可以形式化陈述以下引理。

引理 1. 给定任务集 $\mathbb{T}$ 的调度 S ，考虑具有 m_i 路并行资源预留系统的任务 $\tau_i \in \mathbb{T}$ 。

设 $G = \tau_{i,j}$ 是任务 τ_i 的一个 DAG 作业实例，包络为 $s_{k_1}, f_{k_1}, \dots, s_{k_p}, f_{k_p}$ 。则对 $q \in \{2, \dots, p\}$ ，从 $f_{k_{q-1}}$ 到 f_{k_q} 区间内完成的工作量具有以下下界：

$$work_i^S(f_{k_{q-1}}, f_{k_q}) \geq serv_i^S(f_{k_{q-1}}, s_{k_q}) + serv_i^S(s_{k_q}, f_{k_q}) - (m_i - 1)len(v_{k_q})$$

其中， v_{k_q} 是包络中在时刻 s_{k_q} 开始、在 f_{k_q} 完成的子作业。

证明: 证明中，我们在时刻 s_{k_q} 将工作量分开，并分别估计 $work_i^S(f_{k_{q-1}}, f_{k_q}) = work_i^S(f_{k_{q-1}}, s_{k_q}) + work_i^S(s_{k_q}, f_{k_q})$ 中的每一项，再合并两项估计，得到所需结果。

首先证明，包络中两个连续子作业的完成与开始之间，DAG 实例会充分利用所提供的服务，即

$$work_i^S(f_{k_{q-1}}, s_{k_q}) = serv_i^S(f_{k_{q-1}}, s_{k_q})$$

对所有 $q \in \{2, \dots, p\}$ 均成立。由于将子作业分发至服务的列表调度具有工作保持性，只要服务可用，就会调度可执行子作业。按定义， s_{k_q} 是 v_{k_q} 能够执行的最早时刻，因此从 $f_{k_{q-1}}$ 到 s_{k_q} 的所有服务，必定都用于处理其他不在包络中的子作业。

其次，我们证明，包络中一个子作业从开始到完成的工作量 $work_i^S(s_{k_q}, f_{k_q})$ ，可估计为

$$\max\{serv_i^S(s_{k_q}, f_{k_q}) - (m_i - 1) \cdot len(v_{k_q}), len(v_{k_q})\}.$$

显然，从 v_{k_q} 开始到完成，至少处理 $len(v_{k_q})$ 的工作量。此外，给定服务 $serv_i^S(s_{k_q}, f_{k_q})$ ，由于 v_{k_q} 串行执行，最多有 $m_i - 1$ 个长度为 $len(v_{k_q})$ 的预留未被使用。因此， $work_i^S(s_{k_q}, f_{k_q}) \geq \max\{serv_i^S(s_{k_q}, f_{k_q}) - (m_i - 1) \cdot len(v_{k_q}), len(v_{k_q})\}$ 。

基于该引理，可以计算 DAG 作业的响应时间。为此，先扩展这一引理。

引理 2. 在引理 1 的条件下，有

$$work_i^S(r_G, r_G + t) \geq serv_i^S(r_G, r_G + t) - (m_i - 1)len(G) \quad (2)$$

成立，其中 r_G 是作业 G 的释放时刻，且 $0 \leq t \leq f_{k_p}$ 。

证明: 引理 1 已经完成了本引理证明的主要部分。只需额外谨慎处理 t 不是包络中时间点的情况。

类似引理 1 的证明，可以得到：对所有 $t \in [f_{k_{q-1}}, s_{k_q}]$ ，有 $work_i^S(f_{k_{q-1}}, t) = serv_i^S(f_{k_{q-1}}, t)$ ；对所有 $t \in [s_{k_q}, f_{k_q}]$ ，有 $work_i^S(s_{k_q}, t) \geq serv_i^S(s_{k_q}, t) - (m_i - 1)len(v_{k_q})$ 。同理，对所有 $t \in [r_G, s_{k_1}]$ ，有 $work_i^S(r_G, t) = serv_i^S(r_G, t)$ 。

将区间 $[r_G, t]$ 分成前面描述的若干部分，并同时估计各部分，即可得到结果。为形式化这一过程，定义

$$\mu := (r_G, s_{k_1}, f_{k_1}, \dots, s_{k_p}, f_{k_p}).$$

对 $q \in \{1, \dots, 2p+1\}$ ，用 $\mu(q)$ 表示 μ 的第 q 个元素，用 $\mu^t(q) := \min\{\mu(q), t\}$ 表示由 t 截断的第 q 个元素。

分解 $work_i^S(r_G, r_G + t)$ ，可以将其写成 $\sum_{q=1}^p work_i^S(\mu^t(2q-1), \mu^t(2q))$ 与 $\sum_{q=1}^p work_i^S(\mu^t(2q), \mu^t(2q+1))$ 的和。第一项的下界，是对应服务值之和 $\sum_{q=1}^p serv_i^S(\mu^t(2q-1), \mu^t(2q))$ ；第二项的下界为 $\sum_{q=1}^p (serv_i^S(\mu^t(2q), \mu^t(2q+1)) - (m-1)len(v_{k_q}))$ 。合并两项结果，得到下界

$$\sum_{q=1}^{2p} serv_i^S(\mu^t(q), \mu^t(q+1)) - (m-1) \left(\sum_{q=1}^p len(v_{k_q}) \right),$$

其下界又为 $serv_i^S(r_G, r_G + t) - (m-1)len(G)$ 。因此， $work_i^S(r_G, r_G + t) \geq serv_i^S(r_G, r_G + t) - (m-1)len(G)$ 。

定义 12 (服务界函数). 对任务 $\tau_i \in \mathbb{T}$ ，资源预留系统在长度为 $t \geq 0$ 的区间内提供的最小服务，记为 $sbfi(t)$ 。我们将 $sbfi$ 称为任务 τ_i 的服务界函数。

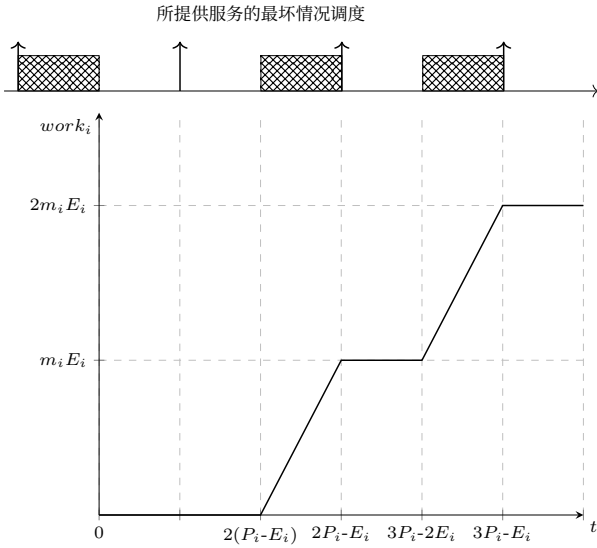


图 3. 资源预留系统的供给界函数 $sbf_i(t)$ 。

利用服务界函数, 对于所有调度 S , 都有下界 $serv_i^S(r_G, r_G + t) \geq sbf_i(t)$ 。由此得到以下定理。

定理 1 (响应时间界). 考虑任务 $\tau_i \in \mathbb{T}$ 。假设任务 τ_i 的预留系统具有 m_i 路并行性, 其最小服务由 sbf_i 描述。设 G 是描述任务 τ_i 实例 $\tau_{i,j}$ 的 DAG, 则 G 的响应时间具有以下上界:

$$\min\{t > 0 \mid sbf_i(t) \geq W_i^G\}. \quad (3)$$

其中, 为简化记号, $W_i^G := vol(G) + (m_i - 1) \cdot len(G) + backlog_i^S(r_G)$ 。

证明: 设 $t' := \min\{t > 0 \mid sbf_i(t) \geq W_i^G\}$ 。采用反证法: 假设 t' 不是响应时间上界, 则 $t' < f_{k_p}$, 其中 f_{k_p} 是 G 包络的最后一个元素。

在这种情况下, 引理 2 给出:

$$\begin{aligned} work_i^S(r_G, r_G + t') &\geq serv_i^S(r_G, r_G + t') - (m_i - 1)len(G) \\ &\geq sbf_i(t') - (m_i - 1)len(G) \end{aligned}$$

按 t' 的定义, 有 $sbf_i(t') \geq vol(G) + (m_i - 1) \cdot len(G) + backlog_i^S(r_G)$ 。因此,

$$work_i^S(r_G, r_G + t') \geq vol(G) + backlog_i^S(r_G)$$

作业 G 在时刻 t' 已完成, 即 $t' \geq f_{k_p}$ 。■

我们强调, 不要求预留方案及其供给界函数遵循特定种类的资源预留方案。响应时间计算的复杂度只取决于供给界函数。例如, 图 3 展示定义 8 中本文预留系统的供给界函数; 如图所示, 最坏情况下, 任务最多可能在 $2(P_i - E_i)$ 个时间单位内得不到服务。

第一次资源预留激活必须不晚于任务 τ_i 第一个作业的释放, 否则分析无效。不过, 只要当新作业进入就绪队列时重新开始分配预留, 系统就可以在任务 τ_i 没有积压或未完成作业时, 停止分配新的预留服务器。

如果采用定义 8 中的预留服务器, 则 DAG 作业 G 的响应时间或服务时间可由以下定理描述。

定理 2 (服务时间). 设 $G = \tau_{i,j}$ 是任务 τ_i 的一个实例。假设对任务 τ_i , 采用定义 8 中的预留系统, 包含 m_i 个同样大小的并行服务, 满足 $E_i \leq P_i$ 。我们可以给出 G 的响应时间上界 R_G :

$$R_G = \left(\left\lceil \frac{W_i^G}{m_i E_i} \right\rceil + 1 \right) (P_i - E_i) + \frac{W_i^G}{m_i} \quad (4)$$

其中, 为简化记号, $W_i^G := vol(G) + (m_i - 1)len(G) + backlog_i^S(r_G)$ 。

证明: 证明中假设 $vol(G) > 0$, 否则不需要处理任何工作, $R_G = 0$ 已是平凡响应时间界。我们拟使用定理 1, 因此需要找到满足 $sbf_i(t) = W_i^G$ 的最小 $t > 0$ 。下面通过直观说明和形式化证明, 说明此最小 t 正是式 (4) 中的 R_G :

假设具有图 3 所示最坏情况服务。可以看到, 每次提供服务时, 都会同时在 m_i 个资源上执行。因此, 任务 τ_i 在作业 G 完成之前, 所需服务总时长为 $\frac{W_i^G}{m_i}$; 该服务跨越 $\left\lceil \frac{W_i^G}{m_i E_i} \right\rceil + 1$ 个服务周期。因此, 必须加上同样次数的无服务时长, 即任务 τ_i 在每个服务周期中不获服务的 $(P_i - E_i)$ 。总体响应时间为 $\left(\left\lceil \frac{W_i^G}{m_i E_i} \right\rceil + 1 \right) (P_i - E_i) + \frac{W_i^G}{m_i}$ 。

形式化证明中, 同样假设图 3 中的最坏情况服务。对于函数 $g: \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0}$, 定义

$$g(t) := \left(\left\lceil \frac{t}{m_i E_i} \right\rceil + 1 \right) (P_i - E_i) + \frac{t}{m_i}$$

复合映射 $sbf \circ g$ 是恒等映射; 函数 g 选择 $sbf_i(t)$ 原像中的最小值, 即 $g(t) = \min(sbf_i^{-1}(t))$ 成立。因此得到 $g(W_i^G) = \min\{t > 0 \mid sbf_i(t) \geq W_i^G\}$ 。■

一般而言, 若已知前一个作业积压工作量的上界 b , 就可以将式 (4) 的响应时间界表示为不依赖先前调度的形式:

$$R'_G(b) = \left(\left\lceil \frac{V_i^G(b)}{m_i E_i} \right\rceil + 1 \right) (P_i - E_i) + \frac{V_i^G(b)}{m_i} \quad (5)$$

其中, $V_i^G(b) := vol(G) + (m_i - 1)len(G) + b$ 。根据式 (5), 我们分别为前一个作业错过截止时间, 以及前一个作业未错过截止时间的情况, 给出响应时间界。

推论 1. 在定理 2 的假设下, 若前一个作业错过截止时间, 则 $R'_G(\rho_i \cdot m_i)$ 是 G 响应时间的上界; 若前一个作业未错过截止时间, 则 $R'_G(0)$ 是响应时间上界。

证明: 这直接来自定理 2, 分别使用 $backlog_i^S(r_G) \leq \rho_i \cdot m_i$, 对应错过截止时间, 以及 $backlog_i^S(r_G) = 0$, 对应未错过截止时间。■

V. 资源预留分析与优化

本节设计分析与优化算法, 生成能够在概率意义上可证明地满足连续 k 次错过截止时间概率上界的资源预留系统。我们强调, 为联合设计连续 k 次截止时间错过约束与预留配置, 需要时间高效的算法, 计算任意给定预留配置下连续 k 次错过截止时间的概率。

A. 资源预留系统分析

基于概率条件 DAG 任务 τ_i 的 DAG 结构 G 的有限样本空间, 定义随机变量 $R_i^1 := (G \mapsto R'_G(\rho_i m_i))$ 和 $R_i^0 := (G \mapsto R'_G(0))$; 它们分别为每个 DAG 作业给出推论 1 中, 此前存在和不存在截止时间错过时的响应时间界。根据定义 7, 若满足下式, 则连续 k 次截止时间错过约束得到满足:

$$\phi_i(0, k) \leq \theta_i(k), \quad (6)$$

其中, $\phi_i(0, k)$ 是任务 τ_i 的前 k 个作业错过截止时间的概率, $\theta_i(k)$ 是预定义值。

由于 $\phi(0, k) = \mathbb{P}(\delta_i(k) > 0, \delta_i(k-1) > 0, \dots, \delta_i(1) > 0)$, 可以使用贝叶斯定理, 将 $\phi(0, k)$ 改写为

$$\mathbb{P}(\delta_i(k) > 0 \mid \delta_i(k-1) > 0, \dots, \delta_i(1) > 0) \cdot \phi_i(k-1).$$

前一个作业的延误增大时, $\tau_{i,k}$ 不满足截止时间的概率不会降低。因此, 若 $\delta_i(k-1) = \rho_i$, 则 $\tau_{i,k}$ 错过截止时间的概率最大。此时, 其他作业的延误 $\delta_i(k-2), \dots, \delta_i(1)$, 与作业 $\tau_{i,k}$ 的延误无关。更具体而言,

$$\begin{aligned} \mathbb{P}(\delta_i(k) > 0 \mid \delta_i(k-1) > 0, \dots, \delta_i(1) > 0) \\ \leq \mathbb{P}(\delta_i(k) > 0 \mid \delta_i(k-1) = \rho_i) \end{aligned}$$

上式成立, 因此可以将连续 k 次错过截止时间的概率界定为

$$\phi_i(0, k) \leq \mathbb{P}(\delta_i(k) > 0 \mid \delta_i(k-1) = \rho_i) \cdot \phi_i(0, k-1). \quad (7)$$

由推论 1 可知,

$$\mathbb{P}(\delta_i(k) > 0 \mid \delta_i(k-1) = \rho_i) \leq \mathbb{P}(R_i^1 > D_i)$$

对于第一个作业, 此前未错过截止时间, 其概率为

$$\phi_i(0, 1) = \mathbb{P}(\delta_i(1) > 0) \leq \mathbb{P}(R_i^0 > D_i).$$

合并结果, 得到连续 k 次错过截止时间的概率界:

$$\begin{aligned} \phi_i(0, k) &\leq \mathbb{P}(R_i^1 > D_i) \cdot \phi_i(0, k-1) \\ &\leq \dots \leq \mathbb{P}(R_i^1 > D_i)^{k-1} \cdot \phi_i(0, 1) \\ &\leq \mathbb{P}(R_i^1 > D_i)^{k-1} \cdot \mathbb{P}(R_i^0 > D_i) \end{aligned}$$

由于 $\mathbb{P}(R_i^0 > D_i) \leq \mathbb{P}(R_i^1 > D_i)$, 还可以得到任务 τ_i 连续 k 次错过截止时间概率的简化上界:

$$\phi_i(0, k) \leq \mathbb{P}(R_i^1 > D_i)^k. \quad (8)$$

为推导排队系统的响应时间上界, 必须先证明系统稳定。非形式化地说, 稳定意味着预留系统的所有积压工作量最终都会在某个时刻处理完毕。我们先形式化定义稳定性, 再证明所设计的基于预留的排队系统在构造上就是稳定的。

定义 13 (稳定性). 若对于所有满足 $\delta_i(\ell) = 0$ 的 $\ell \geq 0$, 几乎必然存在 $k > 0$, 使 $\delta_i(k + \ell) = 0$, 则称资源预留系统 $\mathcal{R}_i$ 稳定。更形式化地说,

$$\lim_{k \rightarrow \infty} \phi(0, k) = 0, \quad (9)$$

即连续 k 次错过截止时间的概率, 在 $k \rightarrow \infty$ 时趋于 0。■

算法 1 资源预留系统的计算

输入 $\mathbb{T}, \theta_1(k_1), \theta_2(k_2), \dots, \theta_n(k_n), \Omega_1, \Omega_2, \dots, \Omega_n$;
输出 满足上述要求的 $\mathcal{R}_1, \mathcal{R}_2, \dots, \mathcal{R}_n$;

```

1: 初始化预留  $\mathcal{R} \leftarrow \{\}$ ;
2: for 集合  $\{\tau_1, \tau_2, \dots, \tau_n\}$  中的每个任务  $\tau_i$  do
3:   for  $\{1, 2, \dots, \Omega_i\}$  中的每个  $m_i$  do
4:      $E_i \leftarrow \min\{E_i \mid (\Phi_i^n)^{k_i} \leq \theta_i(k_i)\}$ ;
5:     if 无法找到  $E_i$  then
6:       continue;
7:     else
8:        $\mathcal{R}_i \leftarrow \mathcal{R}_i \cup \{m_i \text{ 个服务量为 } E_i\}$  的预留;
return  $\mathcal{R}$ ;
```

定理 3 (稳定性). 若 $\mathbb{P}(R_i' > D_i) < 1$, 则资源预留系统 $\mathcal{R}_i$ 稳定。

证明: 根据式 (8), 连续 k 次错过截止时间的概率上界为 $\phi_i(0, k) \leq \mathbb{P}(R_i^1 > D_i)^k$ 。若 $(R_i^1 > D_i) < 1$, 则当 $k \rightarrow \infty$ 时, $\mathbb{P}(R_i^1 > D_i)^k \rightarrow 0$ 。定理得证。■

因此, 在设计资源预留系统时, 除连续 k 次截止时间错过约束外, 无需再单独考虑稳定性。

B. 分布函数计算

本节说明如何实际计算响应时间上界。首先定义辅助随机变量

$$X_i := \frac{\text{vol}(G) + (m_i - 1) \cdot \text{len}(G) + \rho_i \cdot m_i}{m_i \cdot E_i} = \frac{V_i^G}{m_i E_i}$$

其分布函数 $\mathbb{P}(X_i \leq u)$ 可以直接从概率 DAG 任务模型计算, 即如前所述, 枚举所有可能 DAG 作业结构, 并用其实现概率加权。根据推论 1, R_i^1 的分布函数可写为:

$$\mathbb{P}(R_i^1 \leq u) = \mathbb{P}((P_i - E_i) \cdot (\lceil X_i \rceil + 1) + E_i \cdot X_i \leq u)$$

设 $\text{dom}(X_i)$ 表示 X_i 的所有可能取值, 再定义常数集合 $I_i := \{\ell \in \mathbb{N} \mid \lfloor \inf(\text{dom}(X_i)) \rfloor \leq \ell \leq \lceil \sup(\text{dom}(X_i)) \rceil\}$ 。给定 I_i , $\psi(X_i) = (P_i - E_i) \cdot (\lceil X_i \rceil + 1) + E_i \cdot X_i$ 的定义域可以划分为:

$$\bigcup_{\ell \in I_i} \{(P_i - E_i) \cdot (\ell + 2) + E_i \cdot X_i \mid \ell < X_i \leq \ell + 1\}$$

因为对每个 $X_i \in (\ell, \ell + 1]$, 均有 $\lceil X_i \rceil \mapsto \ell + 1$ 。利用分布函数的 σ 可加性并重新整理, 得到

$$\sum_{\ell \in I_i} \mathbb{P}(X_i \leq \frac{u - (P_i - E_i) \cdot (\ell + 2)}{E_i} \mid \ell < X_i \leq \ell + 1) \quad (10)$$

C. 资源预留系统优化

本节给出算法 1, 计算用于调度概率受限截止时间条件 DAG 任务的资源预留系统。给定可容忍连续 k_i 次截止时间错过概率的上界以及延误上界, 目标是找到最少并行预留数量 m_i 及对应的最小服务时长 E_i 。对于每个概率受限截止时间条件 DAG 任务, 算法遍历并行预留数量

$m_i \in [1, \Omega_i]$, 搜索仍满足连续截止时间错过约束所需的最小预留服务量, 确定所有可行配置 (m_i, E_i) 。

定理 4 (单调性). 函数

$$\Phi_i^n : \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0}, E_i \mapsto \mathbb{P}(R_i^1 > D_i)_{|m_i=n}$$

在固定并行预留数量下, 给出截止时间错过概率上界; 它关于服务时长 E_i 单调递减。

证明: 为便于阅读, 记

$$Y_i := \frac{\text{vol}(G_i) + (m_i - 1) \cdot \text{len}(G_i) + \rho_i \cdot m_i}{m_i}$$

对于每个固定 m_i , 其分布函数不依赖于 E_i 。根据本节开头对 $\mathbb{P}(R_i^1 > D_i)$ 的定义, 需要证明

$$\begin{aligned} & \mathbb{P}\left(\left(\left\lceil \frac{Y_i}{E_i} \right\rceil + 1\right) \cdot (P_i - E_i) + Y_i > D_i\right) \\ & \geq \mathbb{P}\left(\left(\left\lceil \frac{Y_i}{E_i + \delta} \right\rceil + 1\right) (P_i - (E_i + \delta)) + Y_i > D_i\right) \end{aligned}$$

对任意非负增量 $\delta \geq 0$, 以及 $Y_i \geq 0$ 的任意实现, 均成立。设任意实现 $Y_i \geq 0$ 满足

$$\left(\left\lceil \frac{Y_i}{E_i + \delta} \right\rceil + 1\right) \cdot (P_i - (E_i + \delta)) + Y_i > D_i$$

此时, Y_i 同样满足

$$\left(\left\lceil \frac{Y_i}{E_i} \right\rceil + 1\right) \cdot (P_i - E_i) + Y_i > D_i$$

, 因此根据分布函数的性质, 所需结论成立。■

由于引理 4 展示了函数 Φ_i^n 的单调性, 可以在区间 $(0, D_i]$ 中使用二分搜索, 找到保证连续截止时间错过约束所需的最小预留服务量。我们强调, Ω_i 是用户指定上界, 可以设置为大于可用处理器数量的任意固定值; 也可以设为一个转折点: 继续增加并行预留数量, 已无法显著降低满足截止时间错过概率约束所需的服务量。

VI. 结论与未来工作

本文为广泛使用的条件并行 DAG 任务模型提出概率版本及形式化描述, 并提出能够进行无调度异常调度的资源预留系统, 同时可证明地保证有界延误、稳定性和连续 k 次截止时间错过概率上界等概率量。此外, 我们提供算法, 针对这些量优化预留系统, 并表明在允许错过截止时间的情况下, 具有高并行度的概率条件 DAG 任务能够改善系统资源使用。未来, 我们计划进一步提高所提出界的紧致性, 并通过实现原型系统评估方法的有效性。

致谢

本研究获得欧洲研究理事会(ERC)2019 年 Consolidator Award 资助, 属于 PropRT 项目, 编号 865170; 同时获得德国研究基金会 (DFG) Sus-Aware 项目资助, 项目编号 398602212。

参考文献

- [1] S. Altmeyer, R. Douma, W. Lunniss, and R. I. Davis. Outstanding paper: Evaluation of cache partitioning for hard real-time systems. In *2014 26th Euromicro Conference on Real-Time Systems*, pages 15–26, 2014.
- [2] S. Baruah. Federated scheduling of sporadic DAG task systems. In *IEEE International Parallel and Distributed Processing Symposium, IPDPS*, pages 179–186, 2015.
- [3] S. Baruah. The federated scheduling of systems of conditional sporadic DAG tasks. In *Proceedings of the 15th International Conference on Embedded Software (EMSOFT)*, 2015.
- [4] S. Ben-Amor, L. Cucu-Crosjean, and D. Maxim. Worst-case Response Time Analysis for Partitioned Fixed-Priority DAG tasks on identical processors. In *IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1423–1426, 2019.
- [5] S. Ben-Amor, L. Cucu-Grosjean, M. Mezouak, and Y. Sorel. Probabilistic Schedulability Analysis for Real-time Tasks with Precedence Constraints on Partitioned Multi-core. In *IEEE International Symposium on Real-Time Distributed Computing (ISORC)*, pages 142–143, 2020.
- [6] V. Bonifaci, A. Marchetti-Spaccamela, S. Stiller, and A. Wiese. Feasibility analysis in the sporadic dag task model. In *ECRTS*, pages 225–233, 2013.
- [7] G. v. d. Brüggen, N. Piatkowski, K.-H. Chen, J.-J. Chen, and K. Morik. Efficiently Approximating the Probability of Deadline Misses in Real-Time Systems. In *30th Euromicro Conference on Real-Time Systems (ECRTS)*, 2018.
- [8] D. Casini, A. Biondi, G. Nelissen, and G. Buttazzo. Partitioned Fixed-Priority Scheduling of Parallel Tasks Without Preemptions. In *IEEE Real-Time Systems Symposium (RTSS)*, pages 421–433, 2018.
- [9] K.-H. Chen, G. v. d. Brüggen, and J.-J. Chen. Analysis of Deadline Miss Rates for Uniprocessor Fixed-Priority Scheduling. In *The 24th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2018.
- [10] P. Chen, W. Liu, X. Jiang, Q. He, and N. Guan. Timing-anomaly free dynamic scheduling of conditional dag tasks on multi-core systems. *ACM Trans. Embed. Comput. Syst.*, 18(5s), Oct. 2019.
- [11] H. S. Chwa, J. Lee, K. Phan, A. Easwaran, and I. Shin. Global EDF Schedulability Analysis for Synchronous Parallel Tasks on Multicore Platforms. In *Euromicro Conference on Real-Time Systems, ECRTS*, pages 25–34, 2013.
- [12] M. E. Conway. A Multiprocessor System Design. In *Proceedings of the November 12-14, 1963, Fall Joint Computer Conference, AFIPS '63 (Fall)*, page 139–146. Association for Computing Machinery, 1963.
- [13] G. Fernandez, J. Abella, E. Quiñones, C. Rochange, T. Vardanega, and F. J. Cazorla. Contention in Multicore Hardware Shared Resources: Understanding of the State of the Art. In *14th International Workshop on Worst-Case Execution Time Analysis, WCET*, volume 39, 2014.
- [14] J. Fonseca, G. Nelissen, and V. Nélis. Improved Response Time Analysis of Sporadic DAG Tasks for Global FP Scheduling. In *Proceedings of the 25th International Conference on Real-Time Networks and Systems*, 2017.
- [15] J. C. Fonseca, G. Nelissen, V. Nélis, and L. M. Pinho. Response time analysis of sporadic DAG tasks under partitioned scheduling. In *11th IEEE Symposium on Industrial Embedded Systems, SIES*, pages 290–299.
- [16] J. Goossens and V. Berten. Gang FTP scheduling of periodic and parallel rigid real-time tasks. *CoRR*, abs/1006.2617, 2010.
- [17] C. Hobbs, Z. Tong, and J. H. Anderson. Optimal soft real-time semi-partitioned scheduling made simple (and dynamic). In *Proceedings of the 27th International Conference on Real-Time Networks and Systems, RTNS*, pages 112–122, 2019.
- [18] K. Lakshmanan, S. Kato, and R. R. Rajkumar. Scheduling parallel real-time tasks on multi-core processors. In *Proceedings of the 31st IEEE Real-Time Systems Symposium*, pages 259–268, 2010.

- [19] J. P. Lehoczky. Real-time queueing theory. In *17th IEEE Real-Time Systems Symposium*, pages 186–195, 1996.
- [20] J. Li, K. Agrawal, C. Gill, and C. Lu. Federated scheduling for stochastic parallel real-time tasks. In *IEEE 20th International Conference on Embedded and Real-Time Computing Systems and Applications*, pages 1–10, 2014.
- [21] J. Li, J.-J. Chen, K. Agrawal, C. Lu, C. D. Gill, and A. Saifullah. Analysis of federated and global scheduling for parallel real-time tasks. In *26th Euromicro Conference on Real-Time Systems, ECRTS*, pages 85–96, 2014.
- [22] M. Maggio, A. Hamann, E. Mayer-John, and D. Ziegenbein. Control-System Stability Under Consecutive Deadline Misses Constraints. In *32nd Euromicro Conference on Real-Time Systems (ECRTS)*, volume 165, pages 21:1–21:24, 2020.
- [23] C. Maia, M. Bertogna, L. Nogueira, and L. M. Pinho. Response-Time Analysis of Synchronous Parallel Tasks in Multiprocessor Systems. In M. Jan, B. B. Hedia, J. Goossens, and C. Maiza, editors, *22nd International Conference on Real-Time Networks and Systems, RTNS*, page 3, 2014.
- [24] A. Marchetti-Spaccamela, N. Megow, J. Schlöter, M. Skutella, and L. Stougie. On the Complexity of Conditional DAG Scheduling in Multiprocessor Systems. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1061–1070. IEEE, 2020.
- [25] A. Melani, M. Bertogna, V. Bonifaci, A. Marchetti-Spaccamela, and G. C. Buttazzo. Response-Time Analysis of Conditional DAG Tasks in Multiprocessor Systems. In *Proceedings of the 2015 27th Euromicro Conference on Real-Time Systems*, 2015.
- [26] M. Nasri, G. Nelissen, and B. B. Brandenburg. Response-Time Analysis of Limited-Preemptive Parallel DAG Tasks Under Global Scheduling. In *31st Euromicro Conference on Real-Time Systems (ECRTS)*, pages 21:1–21:23, 2019.
- [27] L. Palopoli, D. Fontanelli, L. Abeni, and B. V. Frias. An analytical solution for probabilistic guarantees of reservation based soft real-time systems. *IEEE Trans. Parallel Distrib. Syst.*, 27(3):640–653, 2016.
- [28] P. Pazzaglia, C. Mandrioli, M. Maggio, and A. Cervin. DMAC: Deadline-Miss-Aware Control. In *31st Euromicro Conference on Real-Time Systems, ECRTS*, volume 133, pages 1:1–1:24, 2019.
- [29] A. Saifullah, K. Agrawal, C. Lu, and C. Gill. Multi-Core Real-Time Scheduling for Generalized Parallel Task Models. In *Proceedings of the 32nd IEEE Real-Time Systems Symposium*, 2011.
- [30] L. Santinelli, P. M. Yomsi, D. Maxim, and L. Cucu-Grosjean. A component-based framework for modeling and analyzing probabilistic real-time systems. In *IEEE 16th Conference on Emerging Technologies & Factory Automation, ETFA*, pages 1–8, 2011.
- [31] M. A. Serrano, A. Melani, R. Vargas, A. Marongiu, M. Bertogna, and E. Quiñones. Timing characterization of OpenMP4 tasking model. In *International Conference on Compilers, Architecture and Synthesis for Embedded Systems, CASES*, pages 157–166, 2015.
- [32] J. Sun, N. Guan, Y. Wang, Q. He, and W. Yi. Real-time scheduling and analysis of OpenMP task systems with tied tasks. In *IEEE Real-Time Systems Symposium, RTSS*, pages 92–103, 2017.
- [33] N. Ueter, G. von der Brüggen, J. Chen, J. Li, and K. Agrawal. Reservation-based federated scheduling for parallel real-time tasks. In *IEEE Real-Time Systems Symposium (RTSS)*, pages 482–494, 2018.
- [34] H. Yun, G. Yao, R. Pellizzoni, M. Caccamo, and L. Sha. MemGuard: Memory bandwidth reservation system for efficient performance isolation in multi-core platforms. In *2013 IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 55–64, 2013.